

**Procesamiento de Lenguaje Natural con
Aprendizaje Profundo,
Máster en Ciencia y Tecnología Informática
Curso 2025-2026**

**Tema 2: Redes convolucionales para
Procesamiento de Lenguaje Natural**

Objetivos

- Conocer los fundamentos de las redes convolucionales.
- Comprender las operaciones de convolución y pooling.
- Saber cómo aplicar redes convolucionales pueden ser aplicadas a tareas de PLN.

Índice

- Redes convolucionales.
- Operación de convolución.
- Operación de pooling.
- Redes convolucionales para PLN.
- Resumen

¿Qué partes de la imagen permiten identificar un pájaro?



Image by [David Mark](#) from [Pixabay](#)

¿Qué partes de la imagen permiten identificar un pájaro?

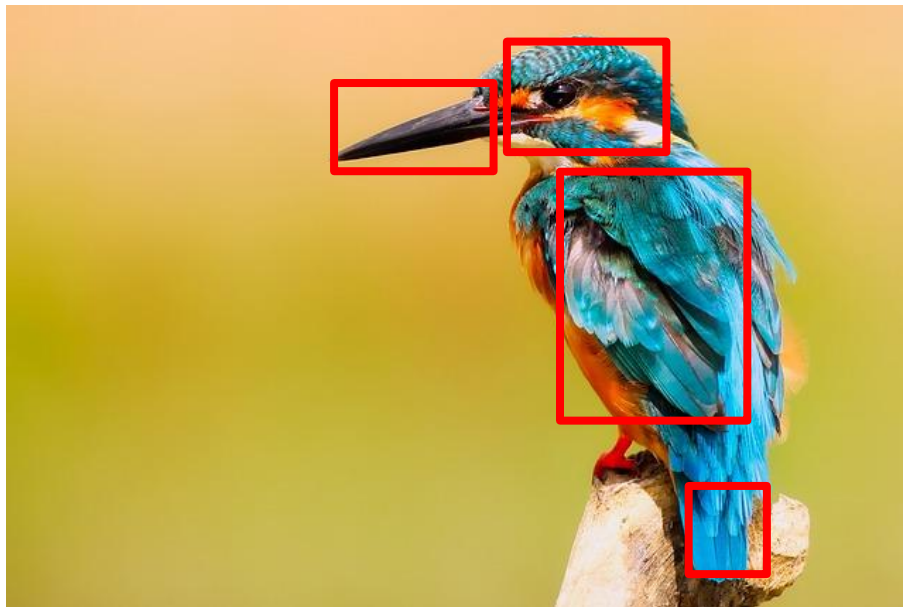


Image by [David Mark](#) from [Pixabay](#)

Redes convolucionales

- En inglés, Convolutional Neural Network (CNN)
- Propuestas originariamente en sistemas de clasificación de imágenes.
- **Idea principal:**
 - En una imagen, no todas las regiones aportan la misma información
 - Algunas partes más pequeñas son especialmente relevantes para identificar el objeto de la imagen.

Redes convolucionales

Estos filtros (o patrones locales) pueden tener diferentes tamaños y diferentes localizaciones. ¿Cómo podemos encontrarlos?

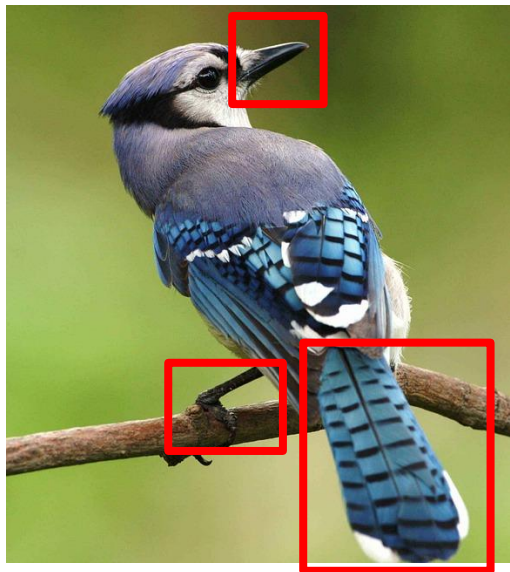


Image by [PublicDomainImages](#) from [Pixabay](#)

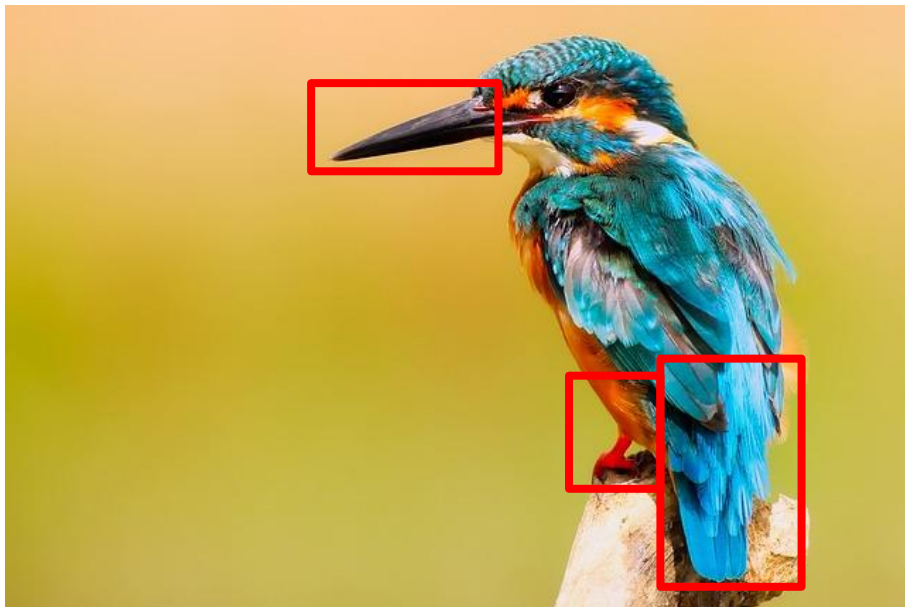


Image by [David Mark](#) from [Pixabay](#)

¿Cómo se detectan esos patrones?:

- En las CNN, los patrones se detectan mediante filtros convolucionales
- Se utilizan múltiples filtros, generalmente de distintos tamaños
- Los filtros se desplazan por toda la imagen

Durante el entrenamiento, la red aprende automáticamente:

- qué filtros son más relevantes
- y cómo combinarlos para clasificar la imagen

Representación de una imagen como entrada de una red neuronal

- Una imagen se representa como un tensor de valores numéricos (una matriz en el caso más simple).

¿Qué es un tensor?

- Estructura de datos numérica multidimensional
- Generaliza vectores y matrices
- Es la representación básica en deep learning

Todo lo que procesa (entrada, salida y pesos) una red neuronal es un tensor.

Representación de una imagen como entrada de una red neuronal

- Una imagen se representa como un tensor de valores numéricos (una matriz en el caso más simple).
- Cada valor corresponde a la intensidad de un píxel:
 - a. En blanco y negro: 1 valor.
 - b. En color: 3 valores (RGB).

Por tanto, una imagen se representa como un tensor:

- imágenes en blanco y negro (tensor 2D): altura $\times$ anchura
- imágenes en color (tensor 3D): altura $\times$ anchura $\times$ 3 (RGB)

Esta representación numérica es la entrada de una red neuronal convolucional.

Representación de una imagen como entrada de una red neuronal

Por ejemplo, la representación de una imagen en color con 5 pixels de altura y 4 de anchura, podría ser:

$$\begin{bmatrix} (255, 0, 0) & (0, 255, 0) & (0, 0, 255) & (255, 255, 0) \\ (0, 255, 255) & (255, 0, 255) & (128, 128, 128) & (64, 64, 64) \\ (0, 0, 0) & (255, 128, 0) & (0, 128, 255) & (128, 0, 128) \\ (200, 50, 50) & (50, 200, 50) & (50, 50, 200) & (180, 180, 60) \\ (30, 30, 30) & (90, 90, 90) & (150, 150, 150) & (220, 220, 220) \end{bmatrix}$$

- **5 filas** → altura
- **4 columnas** → anchura
- Cada píxel tiene **3 valores** (R, G, B)

Representación de una imagen como entrada de una red neuronal

Por ejemplo, la representación de una imagen en blanco y negro con 5 pixels de altura y 4 de anchura, podría ser:

$$\begin{bmatrix} 255 & 180 & 120 & 60 \\ 200 & 150 & 100 & 50 \\ 180 & 130 & 80 & 30 \\ 120 & 90 & 60 & 20 \\ 60 & 40 & 20 & 0 \end{bmatrix}$$

- **5 filas** → altura
- **4 columnas** → anchura
- Cada valor representa la **intensidad del píxel**
 - 0 → negro
 - 255 → blanco

Redes convolucionales. Operaciones:

Las principales operaciones en una red convolucional son:

- convolución.
- pooling.

Convolución

- Una red convolucional utiliza **filtros (kernels)** para analizar la imagen.
 - Se define el número de filtros y su tamaño.
 - Es posible usar filtros de distinto tamaño.
- Cada filtro es una **matriz pequeña de valores numéricos**.
 - Inicialización aleatoria.
 - Valores pequeños.
 - Sus valores se ajustan durante el entrenamiento.
- El filtro se **desplaza por la imagen**, operando sobre regiones locales.
 - en cada desplazamiento, se calcula un valor aplicando *element-wise multiplication*.
- Como resultado se obtiene un mapa de características (feature map).

Operación de convolución

1	0	0	-1
0	3	2	0
4	1	0	2
-2	0	3	1
0	0	1	0

Matriz (entrada)

1	1
1	0

Filtro 2x2

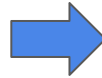
- Cada filtro es una **matriz pequeña de valores numéricos**.
 - Inicialización aleatoria.
 - Valores pequeños.
 - Sus valores se ajustan durante el entrenamiento.

Operación de convolución

1x1	0x1	0	-1
0x1	3x0	2	0
4	1	0	2
-2	0	3	1
0	0	1	0

Matriz (entrada)

1	1
1	0



$$1 \times 1 + 0 \times 1 + 0 \times 1 + 3 \times 0 = 1$$

element-wise multiplication

1		

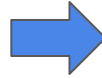
- El filtro se **desplaza por la imagen**, operando sobre regiones locales. En cada desplazamiento, se calcula un valor aplicando *element-wise multiplication*.

Operación de convolución

1	0x1	1x1	-1
0	3x1	2x0	0
4	1	0	2
-2	0	3	1
0	0	1	0

Matriz (entrada)

1	1
1	0



$$0 \times 1 + 1 \times 1 + 3 \times 1 + 2 \times 0 = 4$$

element-wise multiplication

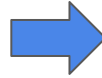
1	4	

Operación de convolución

1	0	1x1	-1x1
0	3	2x1	0x0
4	1	0	2
-2	0	3	1
0	0	1	0

Matriz (entrada)

1	1
1	0



$$1 \times 1 + -1 \times 1 + 2 \times 1 + 0 \times 0 = 2$$

element-wise multiplication

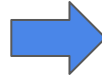
1	4	2

Operación de convolución

1	0	1	-1
0x1	3x1	2	0
4x1	1x0	0	2
-2	0	3	1
0	0	1	0

Matriz (entrada)

1	1
1	0



$$0 \times 1 + 3 \times 1 + 4 \times 1 + 1 \times 0 = 7$$

element-wise multiplication

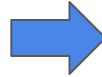
1	4	2
7		

Operación de convolución

1	0	1	-1
0	3x1	2x1	0
4	1x1	0x0	2
-2	0	3	1
0	0	1	0

Matriz (entrada)

1	1
1	0



$$3 \times 1 + 2 \times 1 + 1 \times 1 + 0 \times 0 = 6$$

element-wise multiplication

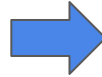
1	4	2
7	6	

Operación de convolución

1	0	1	-1
0	3	2x1	0x1
4	1	0x1	2x0
-2	0	3	1
0	0	1	0

Matriz (entrada)

1	1
1	0



$$2 \times 1 + 0 \times 1 + 0 \times 1 + 2 \times 0 = 2$$

element-wise multiplication

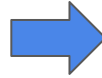
1	4	2
7	6	2

Operación de convolución

1	0	1	-1
0	3	2	0
4x1	1x1	0	2
-2x1	0x0	3	1
0	0	1	0

Matriz (entrada)

1	1
1	0



$$4 \times 1 + 1 \times 1 + -2 \times 1 + 0 \times 0 = 3$$

element-wise multiplication

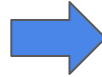
1	4	2
7	6	2
3		

Operación de convolución

1	0	1	-1
0	3	2	0
4	1x1	0x1	2
-2	0x1	3x0	1
0	0	1	0

Matriz (entrada)

1	1
1	0



$$1 \times 1 + 0 \times 1 + 0 \times 1 + 3 \times 0 = 1$$

element-wise multiplication

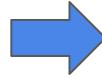
1	4	2
7	6	2
3	1	

Operación de convolución

1	0	1	-1
0	3	2	0
4	1	0x1	2x1
-2	0	3x1	1x0
0	0	1	0

Matriz (entrada)

1	1
1	0



$$0 \times 1 + 2 \times 1 + 3 \times 1 + 1 \times 0 = 5$$

element-wise multiplication

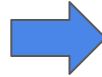
1	4	2
7	6	2
3	1	5

Operación de convolución

1	0	1	-1
0	3	2	0
4	1	0	2
-2x1	0x1	3	1
0x1	0x0	1	0

Matriz (entrada)

1	1
1	0



$$-2 \times 1 + 0 \times 1 + 0 \times 1 + 0 \times 0 = -2$$

element-wise multiplication

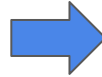
1	4	2
7	6	2
3	1	5
-2		

Operación de convolución

1	0	1	-1
0	3	2	0
4	1	0	2
-2	0x1	3x1	1
0	0x1	1x0	0

Matriz (entrada)

1	1
1	0



$$0 \times 1 + 3 \times 1 + 0 \times 1 + 1 \times 0 = 3$$

element-wise multiplication

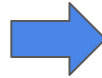
1	4	2
7	6	2
3	1	5
-2	3	

Operación de convolución

1	0	1	-1
0	3	2	0
4	1	0	2
-2	0	3x1	1x1
0	0	1x1	0x0

Matriz (entrada)

1	1
1	0



$$3 \times 1 + 1 \times 1 + 1 \times 1 + 0 \times 0 = 5$$

element-wise multiplication

1	4	2
7	6	2
3	1	5
-2	3	5

Como resultado se obtiene un **mapa de características (feature map)**.

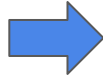
Operación de convolución

1	0	1	-1
0	3	2	0
4	1	0	2
-2	0	3	1
0	0	1	0

Matriz (entrada)

Filtro 2 (1x4)

1	0	0	1
---	---	---	---



Salida (feature map) del
Filtro 2: 5x1

0
0
6
-1
0

Filtros de tamaños distintos, producen mapas de características (feature map) también con tamaños distintos.

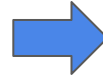
Operación de convolución

1	0	1	-1
0	3	2	0
4	1	0	2
-2	0	3	1
0	0	1	0

Matriz (entrada)

Filtro 3: 5x3

1	0	0
0	0	1
1	0	0
-1	1	1
0	1	0



Salida (feature map) del
Filtro 3: 1x2

12	6
----	---

Filtros de tamaños distintos, producen mapas de características (feature map) también con tamaños distintos.

¿Cuántos filtros se utilizan?

- No existe un número fijo de filtros
- El tamaño del filtro es un **hiperparámetro** que se elige antes del entrenamiento.
- Cada filtro detecta un patrón distinto
- Más filtros = más capacidad (y más coste).

Ejemplos típicos:

- primeras capas: **32, 64 filtros**
- capas profundas: **128, 256 filtros**

Estos valores son orientativos y provienen de la práctica empírica.

¿Tamaño de los filtros?

- El tamaño del filtro es un **hiperparámetro** que se elige antes del entrenamiento.
- Determina la **región local** de la imagen que se analiza en cada paso. Por ejemplo, un filtro de tamaño 3x3 se aplica sobre regiones de 3x3.
- Filtros pequeños capturan **patrones simples y locales**. Son computacionalmente eficiente.
- Filtros grandes capturan **patrones más complejos**. Incrementan el coste computacional
- En la práctica, se utilizan **filtros pequeños**, como: 2x2, **3x3**, 5x5 (imágenes)

Para investigar: stride

- ¿Qué es el **stride** en una convolución?,
- ¿Cómo afecta este parámetro al tamaño de la salida y al coste computacional?

Redes convolucionales. Operaciones:

Las principales operaciones en una red convolucional son:

- convolución.
- **pooling**

Después de aplicar convoluciones

Se obtienen mapas de características que:

- pueden tener un tamaño grande
- contienen información redundante
- incrementan el coste computacional de la red

1	4	2
7	6	2
3	1	5
-2	3	5

0
0
6
-1
0

12	6
----	---

Pooling

Su objetivo es **reducir el tamaño** de los mapas de características (*feature maps*). Se basa en la idea de que:

- no es necesario conservar toda la información espacial exacta
- podemos reducir el tamaño de la representación, manteniendo la información más relevante del objeto

El pooling simplifica la representación sin perder capacidad de reconocimiento.



Image by [PublicDomainImages](#) from [Pixabay](#)



Image by [PublicDomainImages](#) from [Pixabay](#)

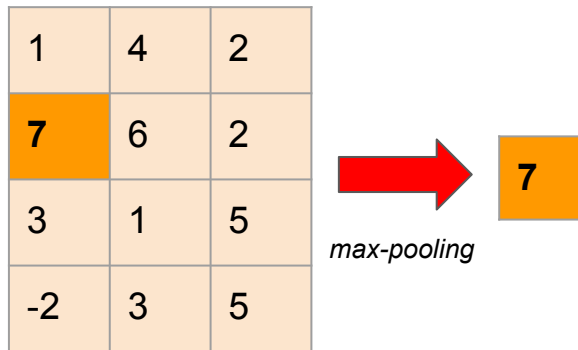
Tipos de pooling

Existen distintas operaciones de pooling que permiten resumir la información local de los mapas de características:

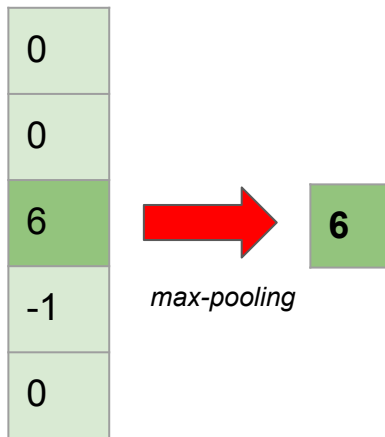
- max pooling: selecciona el valor máximo en cada región. Conserva la activación más fuerte. Es el tipo de pooling más utilizado
- average pooling: calcula el valor medio de cada región. Suaviza la representación. Menos sensible a activaciones extremas
- k-max pooling: selecciona los k valores más altos. Preserva más información que el max pooling. Especialmente útil en PLN

El tipo de pooling influye en qué información se conserva y en el tamaño de la representación.

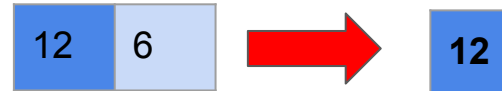
Tipos de pooling - max pooling



Filtro 1
(feature map)

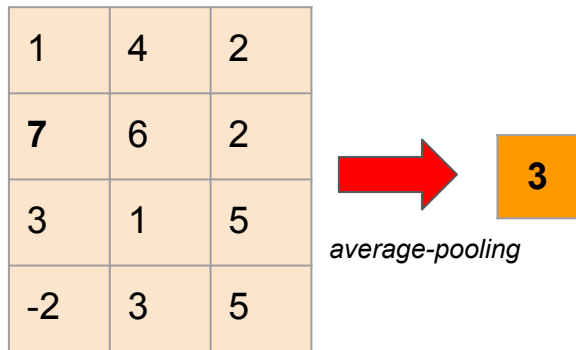


Filtro 2
(feature map)

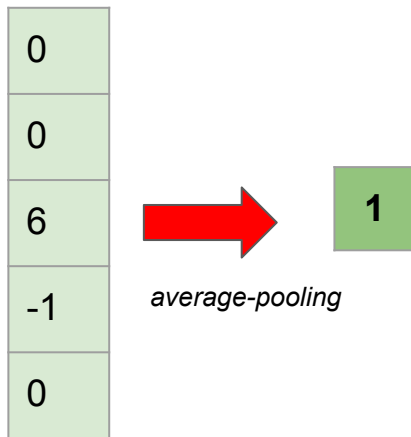


Filtro 3
(feature map)

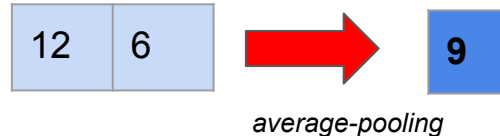
Tipos de pooling - average pooling



Filtro 1
(feature map)



Filtro 2
(feature map)

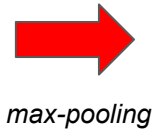


Filtro 3
(feature map)

Tipos de pooling - k-max-pooling

Por ejemplo $k=2$

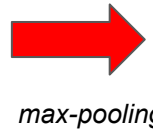
1	4	2
7	6	2
3	1	5
-2	3	5



7	6
---	---

Filtro 1
(feature map)

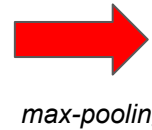
0
0
6
-1
0



6	0
---	---

Filtro 2
(feature map)

12	6
----	---



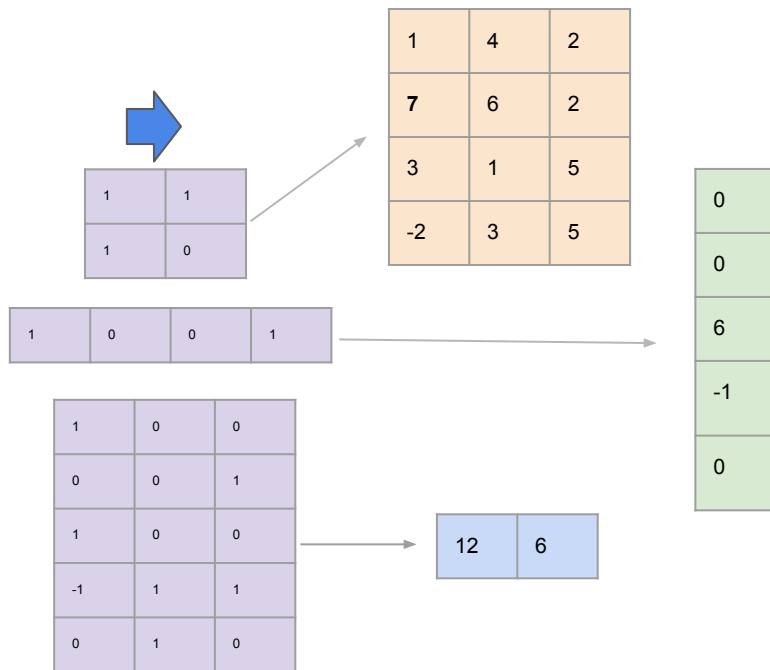
12	6
----	---

Filtro 3
(feature map)

CONVOLUTION

1	0	1	-1
0	3	2	0
4	1	0	2
-2	0	3	1
0	0	1	0

Entrada



Filtros

Feature maps
(Mapa de
características)

POOLING

7	6
6	0
12	6

Salida 2-max-pooling

¿Las redes convolucionales
pueden ser útiles para tareas
de PLN?

Redes convolucionales para PLN

Imagina un sistema de **análisis de sentimiento**, cuyo objetivo es detectar si un texto expresa una opinión positiva, negativa o neutra.

¡Increíble! Esta caja de cereales me dio un desayuno perfectamente equilibrado, como debería ser. Solo me comí la mitad, ¡pero sin duda volveré a comprarla!



Imagen de Дарья Яковлева en Pixabay

¿Es positiva, negativa, o neutra?, ¿por qué?

Redes convolucionales para PLN

Es una opinión positiva

¡Increíble! Esta caja de cereales me dio un desayuno perfectamente equilibrado, como debería ser. Solo me comí la mitad, ¡pero sin duda volveré a comprarla!



Imagen de Дарья Яковлева en Pixabay

Redes convolucionales para PLN

- En una imagen, siempre era posible encontrar pequeñas regiones (filtros) que nos ayudan a clasificar o identificar un objeto.
- De forma similar, en el texto también hay partes más pequeñas que nos van a permitir clasificarlo.
- Por tanto, CNN puede ser útil para tareas de NLP, en especial, para tareas de clasificación de textos.

Representación de un texto como entrada de una red neuronal

- Un **texto** se representa como una **matriz** (tensor 2D) de valores numéricos.
- Cada **fila** representa una **palabra (embedding)**
 - i. Inicialización aleatoria o modelo de word embedding.
 - ii. Todos los embeddings tienen la misma dimensión.
Dimensiones típicas: 50, 100, 200, **300**.
- La matriz tendrá:
 - número de filas de la matriz = número de palabras del texto.
 - número de columnas = dimensión de los embeddings.

Representación de un texto como entrada de una red neuronal

Texto: “*Me gusta la tortilla de patatas*”.

Suponemos que la dimensión de los embeddings es 4.

Una posible representación sería:

Me	0.2	0.1	-0.3	0.4
gusta	0.5	0.2	-0.3	-0.1
la	-0.1	-0.3	-0.2	0.4
tortilla	0.3	-0.3	0.1	0.1
de	0.2	-0.3	0.4	0.2
patatas	0.1	0.2	-0.1	-0.1

Matriz 6x4

Representación de un texto como entrada de una red neuronal

Podemos tener textos de diferentes tamaños.

Por ejemplo: “*Este producto es terrible y no lo recomiendo*” tiene 8 palabras (tokens)

Este	0.1	0.4	0.1	-0.4
producto	0.5	0.2	-0.2	0.1
es	-0.3	0.6	0.2	0.5
terrible	0.5	0.8	0.4	0.6
y	0.4	-0.1	0.3	0.8
no	0.3	0.2	0.2	0.1
lo	0.3	0.7	0.4	0.3
recomiendo	0.5	0.1	0.5	0.2

Matriz 8x4

¿Por qué todas las entradas deben tener el mismo tamaño?

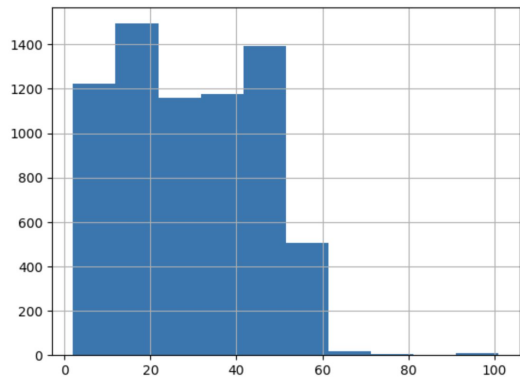
- El dataset de entrenamiento puede contener **textos de distinta longitud**.
- Cada texto se representa como una **matriz de embeddings**.
- Para poder entrenar una red neuronal de forma **eficiente y estable**, todas las entradas deben tener **el mismo tamaño**.
- Esto es necesario porque:
 - el entrenamiento se realiza **por lotes (batches)**
 - las operaciones matemáticas (operaciones con matrices) requieren **dimensiones consistentes**

Por ello, los textos se transforman a una **longitud común**.

¿Cómo logramos que todas las entradas tengan el mismo tamaño?

- En primer lugar, se decide el tamaño óptimo (número de tokens). Opciones:
 - Tamaño máximo: tamaño del texto con mayor número de tokens, o bien,
 - Estudiamos la distribución de tamaño de los textos (tokens), y seleccionamos un tamaño que represente a la mayoría.

Cómo seleccionar el tamaño (número de tokens para representar un texto)



count	6977.000000
mean	28.870861
std	15.884901
min	2.000000
25%	15.000000
50%	28.000000
75%	43.000000
90%	50.000000
95%	53.000000
99%	58.000000
max	101.000000

- Si seleccionamos max=101, es poco eficiente.
- Mejor decisión es 50 (el 90% de los textos tienen como máximo 50 tokens)

¿Cómo logramos que todas las entradas tengan el mismo tamaño?

- En primer lugar, se decide el tamaño óptimo.
- A continuación, debemos asegurarnos que todos los textos tengan ese tamaño.
- Para ello podemos aplicar dos técnicas:
 - **truncation**
 - **padding**

truncation y padding

Padding y truncation permiten que todas las entradas tengan **el mismo tamaño** y puedan procesarse en la red neuronal:

Truncation

- Se **recortan** los textos **con un tamaño > tamaño fijado**.
- Por defecto, se aplica al final.
- Puede implicar pérdida de información

Padding

- Se aplica a textos **más cortos** que el tamaño fijado.
- Se añaden un **token especial** (p. ej., **<PAD>**) hasta alcanzar el tamaño fijado.
Por defecto, al final. Es posible aplicarlo en ambos extremos.
- Permite igualar el tamaño sin añadir información semántica.

padding y truncation

En el ejemplo anterior, si decidimos que el tamaño es 8, debemos aplicar padding en la oración “Me gusta la tortilla de patatas”, para que tenga 8 tokens.

Hemos aplicado padding al principio y al final del texto

<pad>	0	0	0	1
Me	0.2	0.1	-0.3	0.4
gusta	0.5	0.2	-0.3	-0.1
la	-0.1	-0.3	-0.2	0.4
tortilla	0.3	-0.3	0.1	0.1
de	0.2	-0.3	0.4	0.2
patatas	0.1	0.2	-0.1	-0.1
<pad>	0	0	0	1

Convolución en PLN.

- Una red convolucional utiliza **filtros (kernels)** para analizar la texto.
- El filtro se **desplaza por el texto**, operando sobre regiones locales.

Filtros convolucionales en PLN

- En el siguiente ejemplo, estamos aplicando un filtro de tamaño 3.

<pad>	Me	gusta	la	tortilla	de	patatas	<pad>
<pad>	Me	gusta	la	tortilla	de	patatas	<pad>
<pad>	Me	gusta	la	tortilla	de	patatas	<pad>
<pad>	Me	gusta	la	tortilla	de	patatas	<pad>
<pad>	Me	gusta	la	tortilla	de	patatas	<pad>
<pad>	Me	gusta	la	tortilla	de	patatas	<pad>

Filtros convolucionales en PLN

<pad>	0	0	0	1
Me	0.2	0.1	-0.3	0.4
gusta	0.5	0.2	-0.3	-0.1
la	-0.1	-0.3	-0.2	0.4
tortilla	0.3	-0.3	0.1	0.1
de	0.2	-0.3	0.4	0.2
patatas	0.1	0.2	-0.1	-0.1
<pad>	0	0	0	1

- En PLN, el filtro analiza ventanas de palabras.
- Se aplica sobre una secuencia de embeddings (palabras).
- Un filtro es una matriz pequeña de valores aleatorios.
 - su número de filas = tamaño de la ventana
 - su número de columnas = dimensión de los embeddings

Filtros convolucionales en PLN

<pad>	0	0	0	1
Me	0.2	0.1	-0.3	0.4
gusta	0.5	0.2	-0.3	-0.1
la	-0.1	-0.3	-0.2	0.4
tortilla	0.3	-0.3	0.1	0.1
de	0.2	-0.3	0.4	0.2
patatas	0.1	0.2	-0.1	-0.1
<pad>	0	0	0	1

- Ejemplo, si el tamaño de la ventana es 3 tokens, un posible filtro debería ser una matriz de dimensión 3x4 (4 es la dimensión de los embeddings)
- Es un filtro de **tamaño 3**.

3	1	2	-3
-1	2	1	-3
1	1	-1	1

Filtro 1 (tamaño 3)

Convolución en PLN

<pad>	0	0	0	1
Me	0.2	0.1	-0.3	0.4
gusta	0.5	0.2	-0.3	-0.1
la	-0.1	-0.3	-0.2	0.4
tortilla	0.3	-0.3	0.1	0.1
de	0.2	-0.3	0.4	0.2
patatas	0.1	0.2	-0.1	-0.1
<pad>	0	0	0	1

Aplicamos el filtro sobre la entrada:

3	1	2	-3
-1	2	1	-3
1	1	-1	1

Filtro 1 (tamaño 3)

<pad>+me+gusta	-3.6
me+gusta+la	
gusta+la+tortilla	
la+tortilla+de	
tortilla+de+patatas	
de+patatas+pad	

Convolución en PLN

<pad>	0	0	0	1
Me	0.2	0.1	-0.3	0.4
gusta	0.5	0.2	-0.3	-0.1
la	-0.1	-0.3	-0.2	0.4
tortilla	0.3	-0.3	0.1	0.1
de	0.2	-0.3	0.4	0.2
patatas	0.1	0.2	-0.1	-0.1
<pad>	0	0	0	1

Aplicamos el filtro sobre la entrada:

3	1	2	-3
-1	2	1	-3
1	1	-1	1

Filtro 1 (tamaño 3)

<pad>+me+gusta	-3.6
me+gusta+la	1.4
gusta+la+tortilla	
la+tortilla+de	
tortilla+de+patatas	
de+patatas+pad	

Convolución en PLN

<pad>	0	0	0	1
Me	0.2	0.1	-0.3	0.4
gusta	0.5	0.2	-0.3	-0.1
la	-0.1	-0.3	-0.2	0.4
tortilla	0.3	-0.3	0.1	0.1
de	0.2	-0.3	0.4	0.2
patatas	0.1	0.2	-0.1	-0.1
<pad>	0	0	0	1

Aplicamos el filtro sobre la entrada:

3	1	2	-3
-1	2	1	-3
1	1	-1	1

Filtro 1 (tamaño 3)

<pad>+me+gusta	-3.6
me+gusta+la	1.4
gusta+la+tortilla	-0.5
la+tortilla+de	
tortilla+de+patatas	
de+patatas+pad	

Convolución en PLN

<pad>	0	0	0	1
Me	0.2	0.1	-0.3	0.4
gusta	0.5	0.2	-0.3	-0.1
la	-0.1	-0.3	-0.2	0.4
tortilla	0.3	-0.3	0.1	0.1
de	0.2	-0.3	0.4	0.2
patatas	0.1	0.2	-0.1	-0.1
<pad>	0	0	0	1

Aplicamos el filtro sobre la entrada:

3	1	2	-3
-1	2	1	-3
1	1	-1	1

Filtro 1 (tamaño 3)

<pad>+me+gusta	-3.6
me+gusta+la	1.4
gusta+la+tortilla	-0.5
la+tortilla+de	-3,6
tortilla+de+patatas	
de+patatas+pad	

Convolución en PLN

<pad>	0	0	0	1
Me	0.2	0.1	-0.3	0.4
gusta	0.5	0.2	-0.3	-0.1
la	-0.1	-0.3	-0.2	0.4
tortilla	0.3	-0.3	0.1	0.1
de	0.2	-0.3	0.4	0.2
patatas	0.1	0.2	-0.1	-0.1
<pad>	0	0	0	1

Aplicamos el filtro sobre la entrada:

3	1	2	-3
-1	2	1	-3
1	1	-1	1

Filtro 1 (tamaño 3)

<pad>+me+gusta	-3.6
me+gusta+la	1.4
gusta+la+tortilla	-0.5
la+tortilla+de	-3,6
tortilla+de+patatas	-0,2
de+patatas+pad	

Convolución en PLN

<pad>	0	0	0	1
Me	0.2	0.1	-0.3	0.4
gusta	0.5	0.2	-0.3	-0.1
la	-0.1	-0.3	-0.2	0.4
tortilla	0.3	-0.3	0.1	0.1
de	0.2	-0.3	0.4	0.2
patatas	0.1	0.2	-0.1	-0.1
<pad>	0	0	0	1

Aplicamos el filtro sobre la entrada:

3	1	2	-3
-1	2	1	-3
1	1	-1	1

Filtro 1 (tamaño 3)

<pad>+me+gusta	-3.6
me+gusta+la	1.4
gusta+la+tortilla	-0.5
la+tortilla+de	-3,6
tortilla+de+patatas	-0,2
de+patatas+pad	2.0

Convolución en PLN

<pad>	0	0	0	1
Me	0.2	0.1	-0.3	0.4
gusta	0.5	0.2	-0.3	-0.1
la	-0.1	-0.3	-0.2	0.4
tortilla	0.3	-0.3	0.1	0.1
de	0.2	-0.3	0.4	0.2
patatas	0.1	0.2	-0.1	-0.1
<pad>	0	0	0	1

Aplicamos el filtro sobre la entrada:

3	1	2	-3
-1	2	1	-3
1	1	-1	1

Filtro 1 (tamaño 3)

<pad>+me+gusta	-3.6
me+gusta+la	1.4
gusta+la+tortilla	-0.5
la+tortilla+de	-3,6
tortilla+de+patatas	-0,2
de+patatas+pad	2.0

Convolución en PLN

<pad>	0	0	0	1
Me	0.2	0.1	-0.3	0.4
gusta	0.5	0.2	-0.3	-0.1
la	-0.1	-0.3	-0.2	0.4
tortilla	0.3	-0.3	0.1	0.1
de	0.2	-0.3	0.4	0.2
patatas	0.1	0.2	-0.1	-0.1
<pad>	0	0	0	1

3	1	2	-3
-1	2	1	-3
1	1	-1	1

Filtro 1 (tamaño 3)

**Feature map
del Filtro 1**

-3.6
1.4
-0.5
-3.6
-0.2
2.0

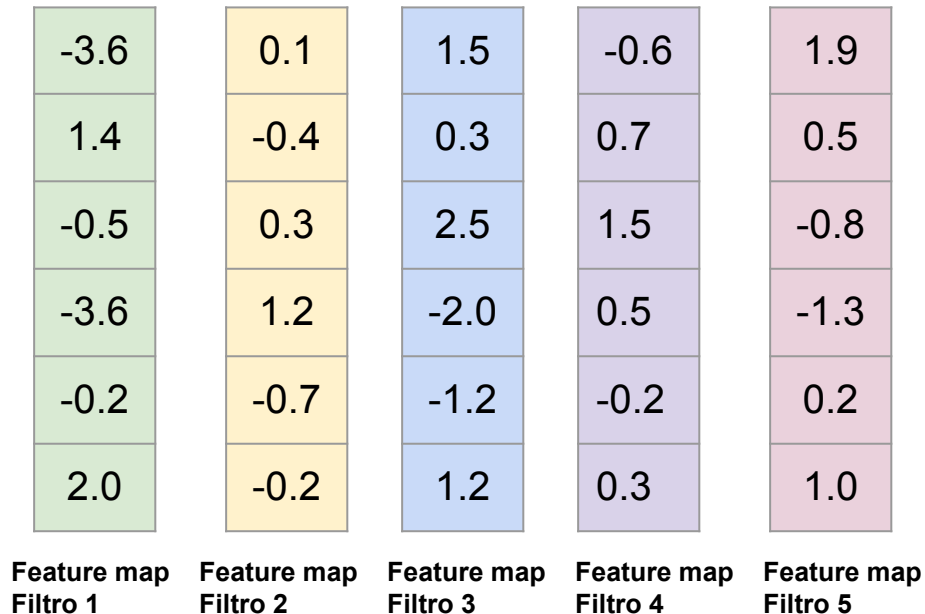
Convolución en PLN. Tamaño y número de filtros

- El tamaño del filtro es un **hiper parámetro** que se elige antes del entrenamiento.
 - Tamaños más comunes: 2-5 tokens.
 - Usar varios tamaños.
 - Mayor tamaño $\leftrightarrow$ mayor coste computacional.
- El número de filtros es otro hiper parámetro:
 - No existe un número fijo.
 - Valores habituales en PLN: 100-300 por cada tamaño de filtro.

Pooling

<pad>	0	0	0	1
Me	0.2	0.1	-0.3	0.4
gusta	0.5	0.2	-0.3	-0.1
la	-0.1	-0.3	-0.2	0.4
tortilla	0.3	-0.3	0.1	0.1
de	0.2	-0.3	0.4	0.2
patatas	0.1	0.2	-0.1	-0.1
<pad>	0	0	0	1

- Supongamos que hemos aplicado 5 filtros (de tamaño 3), y hemos obtenido los siguientes feature maps:



max pooling

<pad>	0	0	0	1
Me	0.2	0.1	-0.3	0.4
gusta	0.5	0.2	-0.3	-0.1
la	-0.1	-0.3	-0.2	0.4
tortilla	0.3	-0.3	0.1	0.1
de	0.2	-0.3	0.4	0.2
patatas	0.1	0.2	-0.1	-0.1
<pad>	0	0	0	1

Aplicamos, por ejemplo, max-pooling:

-3.6	0.1	1.5	-0.6	1.9
1.4	-0.4	0.3	0.7	0.5
-0.5	0.3	2.5	1.5	-0.8
-3.6	1.2	-2.0	0.5	-1.3
-0.2	-0.7	-1.2	-0.2	0.2
2.0	-0.2	1.2	0.3	1.0

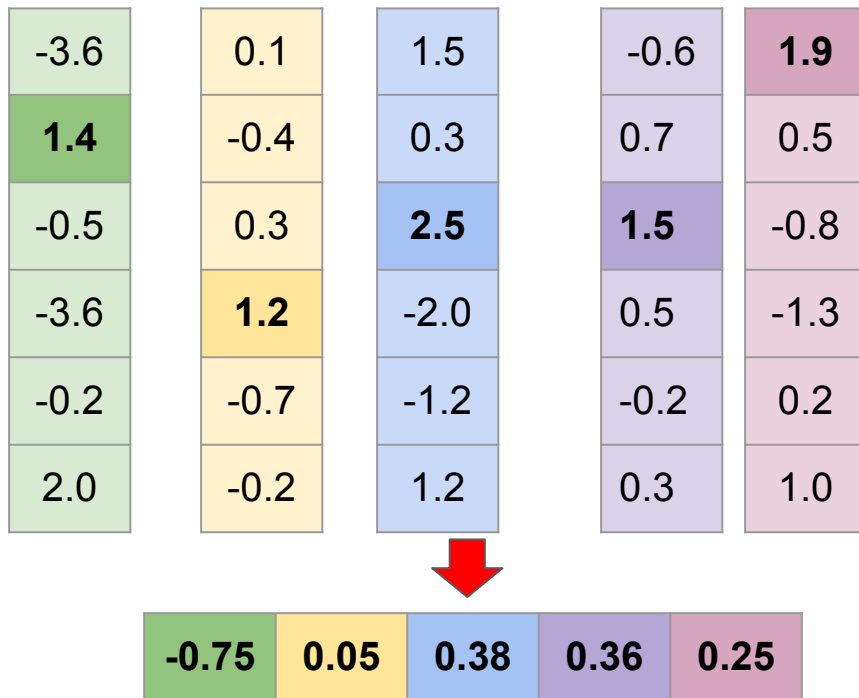


1.4	1.2	2.5	1.5	1.9
------------	------------	------------	------------	------------

average pooling

<pad>	0	0	0	1
Me	0.2	0.1	-0.3	0.4
gusta	0.5	0.2	-0.3	-0.1
la	-0.1	-0.3	-0.2	0.4
tortilla	0.3	-0.3	0.1	0.1
de	0.2	-0.3	0.4	0.2
patatas	0.1	0.2	-0.1	-0.1
<pad>	0	0	0	1

Si aplicamos average-pooling:



CNN en PLN

- Un modelo **CNN** puede contener varias combinaciones de **capas convolucionales** y **pooling**.
- En cada capa convolucional, se define un tamaño de filtro y el número de filtros.

CNN en PLN

- También puede **incluir otras capas** como Dropout, RELU correction layer o una capa densa (fully-connected).
 - **Dropout**: permite ignorar aquellas neuronas cuyos pesos son más bajos que la probabilidad definido en la capa de dropout. Evitan overfitting.
 - **RELU** correction layer ($f(x) = \max(0, x)$), muy simple y rápido de calcular (su derivada es 0 o 1). => mejora los tiempos de entrenamiento.
 - **Capa densa**, suele mejorar los resultados en tareas de clasificación de textos.

CNN en PLN

- La **última capa** de la red CNN dependerá de la tarea que se está resolviendo.
- Por ejemplo:
 - En **clasificación binaria**, esta capa debería ser la función [sigmoid](#). Esta función devuelve un valor entre 0 y 1, que indica la probabilidad de que la entrada pertenezca a la clase positiva.
 - En **multi-clasificación**, esta capa debería estar formada por **neuronas softmax** (tantas neuronas como clases). La función [softmax](#) devuelve la probabilidad por cada clase. Entonces la red devuelve la **clase con mayor probabilidad**.

Implementación en Keras

- Aquí tienes algunos ejemplos prácticos para implementar modelos CNN para un problema de clasificación de textos
- [Ejemplo clasificación binaria](#)
 - [Versión mejorada](#)
- [Ejemplo multclasificación](#)
- Ejercicio:
 - Modifica la arquitectura de la CNN (puedes añadir otras capas convolucionales, probar distintas funciones de pooling, etc), ¿consigue mejorar los resultados?

Resumen

- Una red convolucional es capaz de detectar de forma automática las regiones más importantes de un objeto sin necesidad de supervisión humana.
- En texto, será capaz de detectar los patrones locales más relevantes para una determinada tarea (por ejemplo, clasificar el texto).
- Más eficiente que otras redes neuronales.
 - Por ejemplo, una red densa tiene muchas más conexiones, y su entrenamiento tendrán una mayor complejidad temporal.
- Además, la operación de pooling reduce aún más la complejidad, tomando los elementos más representativos, y eliminando el resto.

OpenCourseWare
Procesamiento de Lenguaje Natural con
Aprendizaje Profundo,

Gracias!!!

<https://github.com/iseaura>